ชีทเปิดสอบ 1 / แผนที่ API

RESTful API · Express 5 · ESM · MySQL

ใช้กับ spec 67130500001–67130500004 · เฉลยร่วมชุดเดียว พร้อมคำอธิบายและแนวฝึกดัดแปลง อ่านหน้า 3 ก่อนเลือกข้อมูลนักศึกษา

> **ลำดับคิด:** Request → Route → Service → Repository → MySQL → DTO → JSON
> ให้ยึด spec และ schema ของตนเองก่อนชีทนี้เสมอ

Method + path ต่อท้าย /api/รหัส	สิ่งที่ต้องทำ	สถานะ
GET /student-info	คืน studentId, name, secretKey	200
GET /brands	Array ของแบรนด์ทั้งหมด	200
GET /brands/:id	แบรนด์ 1 รายการ / ไม่พบ	200 / 404
GET /products?page=1&limit=10	Summary DTO 7 ฟิลด์ + pagination	200
GET /products/:id	Detail DTO + ชื่อแบรนด์	200 / 404
POST /products	เพิ่มสินค้า / SKU ซ้ำ	201 / 409
PUT /products/:id	แก้ไขสินค้า / ไม่พบ ID	200 / 404
DELETE /products/:id	ลบสำเร็จ ไม่มี body / ไม่พบ	204 / 404

ข้อมูลเข้ามาจากไหน?

```
req.params.id     // /products/7 -> '7'
req.query.page    // /products?page=2 -> '2'
req.body.price    // JSON body: {"price": 29900}
req.pagination    // value created by our middleware
```

Pagination ที่ต้องเปิดเจอทันที

```
offset = (page - 1) * limit
totalPages = Math.ceil(total / limit)
hasNextPage = page * limit < total
hasPrevPage = page > 1
```

ตัวอย่าง: total = 75, page = 3, limit = 10 → offset = 20, totalPages = 8, hasNextPage = true, hasPrevPage = true. ตัวเลข 75 เป็นตัวอย่างใน spec ไม่ใช่ค่าที่ต้อง hardcode

เปิดอ่านส่วนไหนเมื่อทำไม่ออก

ส่วน	หน้า
ชีทเร็ว	1–2
เทียบ spec / ขอบเขต	3–4
Setup	5
เฉลย A–H	6–15
ทดสอบ / แก้ Error / ส่งงาน	16–19
แนวโจทย์ดัดแปลง	20–26
โจทย์ฝึก / คำถาม / checklist / แหล่งอ้างอิง	27–30

[S1–S4] §2.1–2.3; สูตรและการจัดลำดับอ่านเป็นคำอธิบายในชีท

ชีทเปิดสอบ 2 / โค้ดจำเป็นและจุดพลาด

คำสั่งเริ่มงาน

```
npm install
npm start
node tests/test-runner.mjs YOUR_STUDENT_ID
```

คำสั่งสุดท้ายใช้ได้เมื่อได้รับชุด tests จริงจากผู้สอนแล้ว เปิดอีก Terminal ขณะที่ Server ยังทำงานที่ port 3000

SQL 5 รูปแบบหลัก

```
select * from brands order by id;
select * from products where id = ?;
select * from products order by id limit ? offset ?;
insert into products (sku, name) values (?, ?);
update products set price = ? where id = ?;
delete from products where id = ?;
```

INSERT ข้างบนเป็นแม่แบบสั้น ๆ ต้องเติมคอลัมน์บังคับตาม schema จริง ค่าใน array ต้องตรงลำดับเครื่องหมาย ? เสมอ

รูปแบบผลลัพธ์ของ mysql2

```
const [rows] = await db.query(sql, values);
const one = rows[0];             // select 1 row
const [result] = await db.query(insertSql, values);
const newId = result.insertId;   // auto-increment key
const count = result.affectedRows;
```

เช็กก่อนเสียคะแนน

อย่าทำแบบนี้	ให้ทำแบบนี้
ส่งทุกคอลัมน์ใน GET /products	แปลงให้เหลือ Summary DTO 7 ฟิลด์
ใช้จำนวนแถวของหน้าปัจจุบันเป็น total	ใช้ select count(*) นับข้อมูลทั้งหมดที่เข้าเงื่อนไข
ลบสำเร็จแล้วส่ง JSON	res.status(204).end()
if (!data.stock_quantity)	ตรวจ undefined/null; ค่า 0 เป็นข้อมูลที่มีค่า
ใช้ changedRows = 0 แปลว่าไม่พบ ID	ตรวจว่ามีแถวนั้นก่อน update; ค่าเดิมก็สำเร็จได้
คัดลอกชื่อ/Secret Key ของเพื่อน	ใช้ข้อมูลใน spec ที่ตนเองได้รับ
เช็ก SKU ก่อน INSERT อย่างเดียว	ยังต้องมี UNIQUE ที่ฐานข้อมูลเพื่อกันคำขอพร้อมกัน

> **แยก 400 / 404 / 409:** ข้อมูลคำขอไม่ถูกต้อง / ไม่พบ resource / ข้อมูลขัดแย้ง เช่น SKU ซ้ำ ส่วน 500 คือปัญหาฝั่ง Server ที่ต้องอ่าน log

[S1–S4] §2–4; [R2] mysql2 Promise; [R7] HTTP status

ผลเทียบ spec ทั้ง 4 ชุด

ผลตรวจไฟล์จริง: เมื่อแทนรหัสนักศึกษา ชื่อ และ Secret Key ด้วย placeholder แล้ว เนื้อหาทั้ง 4 ไฟล์ตรงกันทั้งหมด ไม่มีชุดใดเพิ่ม endpoint หรือเปลี่ยน DTO จากชุดอื่น

ชุด	ชื่อใน spec	Secret Key
67130500001	Kanokphon Sueby	67KaSu
67130500002	Kritsanaphong	71KrNg
67130500003	Kittikorn Phaen	13KiPh
67130500004	Kittithat Yotkh	30KiYo

> **ใช้ตารางนี้เพื่ออ่านตัวอย่างเท่านั้น:** เอกสารกำชับห้ามใช้ข้อมูลประจำตัวผู้อื่น ในข้อสอบจริงให้เปลี่ยนสามค่าตาม spec ของตนเองเท่านั้น ชีทไม่ถือว่ารหัสตัวอย่างเป็นรหัสของผู้ใช้

คำตอบสำหรับชุดที่เหลือ

```
// Set 00002
const STUDENT_ID = '67130500002';
const STUDENT_NAME = 'Kritsanaphong';
const SECRET_KEY = '71KrNg';

// Set 00003: replace the three lines above
const STUDENT_ID = '67130500003';
const STUDENT_NAME = 'Kittikorn Phaen';
const SECRET_KEY = '13KiPh';

// Set 00004: replace the three lines above
const STUDENT_ID = '67130500004';
const STUDENT_NAME = 'Kittithat Yotkh';
const SECRET_KEY = '30KiYo';
```

เลือกเพียง 1 ชุด: ห้ามวางตัวแปร const ชื่อเดิมทั้ง 3 ชุดพร้อมกันในไฟล์เดียว URL, คำสั่ง test และชื่อ ZIP ต้องใช้รหัสเดียวกันทั้งหมด

คะแนนและสิ่งที่เอกสารไม่ได้ให้

Identity 10 + Brands 10 + Products 65 = **85 คะแนน** ตามหัวข้อที่พิมพ์ไว้ แต่ส่วน Self-Testing บอกเต็ม 100 คะแนน จึงเป็นความไม่ตรงกันในเอกสาร ยังสรุปไม่ได้ว่าอีก 15 คะแนนคืออะไร

ZIP ไม่มี database/init-database.sql และ tests/test-runner.mjs จึงยังยืนยันชื่อตาราง คอลัมน์ constraints หรือผลผ่านชุดตรวจจริงไม่ได้ โค้ดในชีทเป็นเฉลยตัวอย่างตาม JSON และ endpoint ที่ระบุ

[S1–S4] ข้อมูลนักศึกษา, §2.1–2.3, §3–4; เปรียบเทียบไฟล์แบบ normalized และตรวจ ZIP โดยตรง

ก่อนคัดลอกโค้ด / ขอบเขตของเฉลย

เฉลยนี้ใช้โครงสร้าง **Route → Service → Repository → Mapper** ตามแนวโฟลเดอร์ใน template ไม่ใช้ไฟล์ office และ productline เดิม เพราะไม่ใช่ resource ที่ spec นี้ต้องการ

ส่วน	ใน spec ระบุไว้หรือไม่?	สิ่งที่เฉลยเลือกทำ
GET รายการสินค้า	ระบุ Summary DTO + pagination	ส่ง 7 ฟิลด์และ pagination เสมอ
default page	ระบุ 1	ใช้ 1
default limit	ระบุเพียงตัวอย่าง 10	เลือก 10 เพื่อฝึก ไม่ใช่ค่าบังคับที่ยืนยันได้
POST/PUT response body	ไม่กำหนดรูปแบบชัดเจน	เลือก success + Detail DTO + timestamp
PUT ต้องส่งทุก field หรือไม่	ไม่ระบุ	แก้เฉพาะ field ที่ส่งมา; คง field อื่นไว้
Validation / error JSON	ไม่แจกแจงครบ	เพิ่ม 400 และ error.message เพื่อฝึก
ตาราง brands / products, id auto increment	ไม่มี SQL ให้ตรวจ	สมมติชื่อตาม JSON; ต้องตรวจ schema จริง
release_date เป็น DATE	ไม่มี DDL ให้ตรวจ	ใช้ dateStrings: ['DATE'] ในตัวอย่าง
SKU มี UNIQUE	ระบุว่าซ้ำต้อง 409 แต่ไม่มี DDL	ตรวจ UNIQUE จาก schema; ห้ามอ้างว่าในไฟล์มีแล้ว

ไฟล์ที่ต้องสร้างหรือแทนที่

```
src/app.js
src/configs/db-config.js
src/middlewares/error-handler.js
src/middlewares/pagination-middleware.js
src/mappers/api-response.js
src/mappers/product-mapper.js
src/routes/brand-route.js
src/routes/product-route.js
src/services/brand-service.js
src/services/product-service.js
src/repositories/brand-repo.js
src/repositories/product-repo.js
```

> ให้ใช้ไฟล์ในเฉลยชุดนี้ให้ครบ ไม่ผสม product-service หรือ api-response รุ่นเดิมเข้ามาบางส่วน เพราะรูปแบบข้อมูลที่ส่งต่อระหว่างชั้นเปลี่ยนเป็น {rows, total} และ {data, pagination}

[S1–S4] §2; [T1] โครงสร้าง template; แถวที่ระบุ “เลือก/สมมติ” เป็นการออกแบบของเฉลย

Setup / Database / package.json

1. เปิดโฟลเดอร์ที่มี package.json แล้วติดตั้ง

```
npm install
```

ใน package.json เดิมมี "type": "module" อยู่แล้ว เพิ่มคำสั่ง start ใน scripts โดยไม่ลบ dependencies หรือ package-lock.json

```
"scripts": {
  "start": "node src/app.js"
}
```

2. นำเข้า SQL ที่ผู้สอนให้ แล้วตรวจโครงสร้าง

Command Prompt / Bash

```
mysql -u root -p < database/init-database.sql
```

PowerShell

```
cmd /c "mysql -u root -p < database\init-database.sql"
```

ใน MySQL client / Workbench

```
use mobile_distributor;
show tables;
describe brands;
describe products;
show create table products;
show index from products;
```

ดูให้ครบ: primary key ชื่ออะไร, id เพิ่มเองหรือไม่, field ใด not null, default, enum ที่อนุญาต, foreign key และ unique ของ sku อย่ากำหนดรายชื่อ category/status ทั้งหมดจากตัวอย่างเดียว

3. แก้การเชื่อมต่อ

src/configs/db-config.js

```
import mysql from 'mysql2/promise';

const db = mysql.createPool({
  host: process.env.DB_HOST ?? 'localhost',
  port: Number(process.env.DB_PORT ?? 3306),
  user: process.env.DB_USER ?? 'root',
  password: process.env.DB_PASSWORD ?? 'YOUR_MYSQL_PASSWORD',
  database: process.env.DB_NAME ?? 'mobile_distributor',
  dateStrings: ['DATE']
});

export default db;
```

แก้ YOUR_MYSQL_PASSWORD เป็นรหัสผ่านของเครื่อง หรือกำหนด DB_PASSWORD ใน environment ตัวอย่างนี้ไม่อ่านไฟล์ .env อัตโนมัติ dateStrings เลือกให้เฉพาะชนิด DATE กลับมาเป็น YYYY-MM-DD โดยไม่แปลงเขตเวลา

> ถ้าไม่มี SQL จริง ให้ใช้ practice/practice-only.sql เฉพาะฝึกกับฐานข้อมูล mobile_distributor_practice ที่แยกไว้ ห้ามนำ schema จำลองไปอ้างว่าเป็น schema ข้อสอบ

[S1–S4] §1.1; [T1] package.json/db-config.js; [R2, R3, R4] mysql2 และ ESM

เฉลย A / app.js และ Student Info

src/app.js

```
import express from 'express';
import brandRouter from './routes/brand-route.js';
import productRouter from './routes/product-route.js';
import { apiResponse } from './mappers/api-response.js';
import { AppError, errorHandler } from './middlewares/error-handler.js';

// Study example 00002. Use ONLY your assigned profile in the real exam.
const STUDENT_ID = '67130500002';
const STUDENT_NAME = 'Kritsanaphong';
const SECRET_KEY = '71KrNg';

const app = express();
app.use(express.json());

app.get(`/api/${STUDENT_ID}/student-info`, (req, res) => {
  return apiResponse(res, {
    studentId: STUDENT_ID,
    name: STUDENT_NAME,
    secretKey: SECRET_KEY
  });
});

app.use(`/api/${STUDENT_ID}/brands`, brandRouter);
app.use(`/api/${STUDENT_ID}/products`, productRouter);
app.use((req, res, next) => next(new AppError('Route not found', 404)));
app.use(errorHandler);

// NODE_ENV=test is used only by our mock-DB study checks.
if (process.env.NODE_ENV !== 'test') {
  app.listen(3000, () => console.log('Server running on port 3000'));
}

export default app;
```

อธิบายส่วนที่ต้องเข้าใจ

express.json() ต้องอยู่ก่อน route ที่รับ JSON เพื่อให้ POST/PUT ใช้ req.body ได้ ส่วน app.use ที่มี base path จะนำ Router มาต่อ URL ให้ครบตามรหัสนักศึกษา

student-info ไม่ใช้ฐานข้อมูล: ใช้ตรวจว่ารหัส ชื่อ และ Secret Key ตรงกับ spec ก่อนเริ่มแก้ SQL เมื่อเปลี่ยนสามตัวแปร base path ทุก resource จะเปลี่ยนตาม

ลำดับท้ายไฟล์: route ทั้งหมด → route-not-found → errorHandler ส่วนเงื่อนไข NODE_ENV มีไว้ให้เครื่องมือทดสอบนำเข้า app โดยไม่เปิด port ซ้ำ; npm start ตามปกติยังฟัง port 3000

```
GET http://localhost:3000/api/67130500002/student-info
```

> timestamp สร้างใหม่ตอนตอบ ไม่คัดลอกเวลา 2026-10-08 ใน JSON ตัวอย่างมาใช้เป็นค่าคงที่

[S2] §1.2 และ §2.1; [R1] Express middleware/error handling; โค้ดเขียนขึ้นสำหรับชีท

เฉลย B / Response และ Error Handler

src/mappers/api-response.js

```
export function apiResponse(res, data, pagination = null, status = 200) {
  const body = { success: true, data };
  if (pagination !== null) body.pagination = pagination;
  body.timestamp = new Date().toISOString();
  return res.status(status).json(body);
}
```

src/middlewares/error-handler.js

```
export class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

export function errorHandler(err, req, res, next) {
  if (res.headersSent) return next(err);

  let status = err.statusCode ?? err.status ?? 500;
  let message = err.message;

  if (err.code === 'ER_DUP_ENTRY') {
    status = 409;
    message = 'Duplicate unique value (for example, SKU)';
  } else if (err.code === 'ER_NO_REFERENCED_ROW_2') {
    status = 400;
    message = 'Referenced record does not exist';
  } else if (err.code === 'ER_ROW_IS_REFERENCED_2') {
    status = 409;
    message = 'Resource is in use';
  } else if (err.type === 'entity.parse.failed') {
    status = 400;
    message = 'Invalid JSON body';
  }

  if (status >= 500) {
    console.error(err);
    message = 'Internal server error';
  }

  return res.status(status).json({
    success: false,
    error: { message },
    timestamp: new Date().toISOString()
  });
}
```

apiResponse(res, data, null, 201) ส่งสถานะ Created ส่วน GET และ PUT ใช้ค่าเริ่มต้น 200. DELETE สำเร็จไม่เรียก helper นี้ เพราะ 204 ต้องไม่มี body

Express 5: เมื่อ async route ที่คืน Promise โยน error หรือ await Promise ที่ reject จะส่งต่อเข้า error handler ได้ ฟังก์ชัน error handler ต้องมี 4 พารามิเตอร์ (err, req, res, next)

การแปลง foreign key error เป็น 400/409 และรูปแบบ error.message เป็นทางเลือกของเฉลย ไม่ได้ถูกกำหนดครบใน spec ส่วนข้อความ SQL ภายในของ 500 เก็บใน log ไม่ส่งตรงให้ client

[S1–S4] §2.3 POST/DELETE; [R1] Express 5 error handling; รูปแบบ error ที่เหลือเป็นตัวอย่าง

เฉลย C / Pagination ที่ได้ครบทุก field

src/middlewares/pagination-middleware.js

```
import { AppError } from './error-handler.js';

export function queryParser(req, res, next) {
  const page = Number(req.query.page ?? 1);
  const limit = Number(req.query.limit ?? 10);
  const offset = (page - 1) * limit;

  if (!Number.isSafeInteger(page) || page < 1 ||
      !Number.isSafeInteger(limit) || limit < 1 ||
      !Number.isSafeInteger(offset)) {
    throw new AppError('page and limit must be positive integers', 400);
  }

  req.pagination = { page, limit, offset };
  next();
}
```

ทำไมต้องเขียนแบบนี้?

req.query ได้ค่าจาก URL จึงแปลงด้วย Number ก่อนคำนวณ. ?? 1 ใช้ค่าเริ่มต้นเฉพาะเมื่อไม่ได้ส่งค่า. Number.isSafeInteger ใช้กันทศนิยม ตัวอักษร และจำนวนที่ใหญ่เกินขอบเขตจำนวนเต็มปลอดภัยของ JavaScript

offset คือ **จำนวนรายการที่ข้าม** ไม่ใช่หมายเลขหน้าหรือ id. หน้า 1 ข้าม 0 แถว หน้า 2 ข้าม limit แถว การจัด order by id ทำให้ลำดับของการแบ่งหน้าชัดเจน

total	page	limit	offset	totalPages	next / prev
75	1	10	0	8	true / false
75	3	10	20	8	true / true
75	8	10	70	8	false / true
75	9	10	80	8	false / true
0	1	10	0	0	false / false

เมื่อเกินหน้าสุดท้าย เฉลยเลือกตอบ 200 พร้อม data: [] ไม่ใช่ 404. เมื่อไม่มีข้อมูล สูตรให้ totalPages = 0. สองกรณีนี้เป็นพฤติกรรมที่เลือกจากสูตร ไม่ใช่รายละเอียดที่ spec แจกแจงไว้

ข้อผิดพลาดใน middleware เดิมที่ควรแก้

ตัวเดิมคืน pagination = null ถ้าส่ง page หรือ limit ไม่ครบ ทำให้ไม่ตรงแนว default page ของโจทย์ และตอนจำกัด limit สูงสุด กลับคำนวณ offset จาก pageSize เดิม จึงอาจข้ามข้อมูลผิดจำนวน

> เฉลยเรียก queryParser เฉพาะ GET /products ไม่ใช้ app.use(queryParser()) แบบเดิม เพราะ signature ใหม่เป็น middleware โดยตรง ไม่ใช่ฟังก์ชันสร้าง middleware

[S1–S4] §2.3(1); [T1] pagination-middleware.js; [R5] LIMIT/OFFSET; กรณีขอบเขตเป็นนโยบายตัวอย่าง

เฉลย D / DTO และชื่อคอลัมน์

src/mappers/product-mapper.js

```
export default class ProductMapper {
  static toSummaryDTO(p) {
    return {
      id: p.id,
      sku: p.sku,
      name: p.name,
      brandId: p.brand_id,
      category: p.category,
      price: Number(p.price),
      status: p.status
    };
  }

  static toDetailDTO(p) {
    return {
      ...this.toSummaryDTO(p),
      brandName: p.brand_name,
      ramGb: p.ram_gb,
      storageGb: p.storage_gb,
      color: p.color,
      stockQuantity: p.stock_quantity,
      releaseDate: p.release_date
    };
  }
}
```

ค่าจากฐานข้อมูล	ชื่อใน JSON	ใช้ใน
brand_id	brandId	Summary และ Detail
brand_name (alias จาก JOIN)	brandName	Detail
ram_gb / storage_gb	ramGb / storageGb	Detail
stock_quantity	stockQuantity	Detail
release_date	releaseDate	Detail

Summary มี 7 ฟิลด์เท่านั้น: id, sku, name, brandId, category, price, status. ไม่ส่ง stockQuantity หรือ brandName ใน GET รายการ หากโจทย์ยังเป็น spec นี้

Detail: ใช้ ...this.toSummaryDTO(p) เพื่อเอา 7 ฟิลด์เดิมมารวม แล้วเพิ่ม brandName, ramGb, storageGb, color, stockQuantity, releaseDate รวม 13 ฟิลด์ ลำดับ key ใน JSON ไม่ใช่สาระ แต่ชื่อและค่าต้องถูก

Number(p.price): mysql2 ปกติคืน DECIMAL เป็น string จึงแปลงให้ตรง JSON Number ใน spec. ข้อสอบนี้ใช้ราคาขนาดตัวอย่างทั่วไป; ระบบเงินจริงที่ต้องการความเที่ยงตรงสูงต้องออกแบบเรื่องทศนิยมแยกต่างหาก

> releaseDate ใน mapper นี้คาดว่า db-config ตั้ง dateStrings: ['DATE'] และคอลัมน์จริงเป็น DATE. ถ้าชนิดจริงต่างกัน ต้องแก้การอ่านข้อมูลก่อน ไม่ควรตัดวันที่จาก UTC แบบเดาสุ่ม

[S1–S4] §2.3(1)–(2); [R3] DECIMAL และ dateStrings; โค้ด mapper เป็นเฉลย

เฉลย E / Brands API ครบทั้งสองข้อ

src/repositories/brand-repo.js

```
import db from '../configs/db-config.js';

export async function findAll() {
  const [rows] = await db.query('select * from brands order by id');
  return rows;
}

export async function findOne(id) {
  const [rows] = await db.query(
    'select * from brands where id = ?', [id]
  );
  return rows[0];
}
```

src/services/brand-service.js

```
import * as repo from '../repositories/brand-repo.js';
import { AppError } from '../middlewares/error-handler.js';

export async function getAllObjects() {
  return repo.findAll();
}

export async function getObject(id) {
  const brand = await repo.findOne(id);
  if (!brand) throw new AppError('Brand not found', 404);
  return brand;
}
```

src/routes/brand-route.js

```
import express from 'express';
import * as service from '../services/brand-service.js';
import { apiResponse } from '../mappers/api-response.js';

const router = express.Router();

router.get('/', async (req, res) => {
  return apiResponse(res, await service.getAllObjects());
});

router.get('/:id', async (req, res) => {
  return apiResponse(res, await service.getObject(req.params.id));
});

export default router;
```

Repository: ติดต่อ MySQL แล้วคืน rows หรือ rows[0]. **Service:** ตัดสินว่าถ้าไม่มีข้อมูลต้อง 404. **Route:** อ่าน req.params.id และส่ง JSON ผ่าน apiResponse

GET /brands ส่ง array แม้มีแบรนด์เดียว ส่วน GET /brands/1 ส่ง object ของแบรนด์เดียว. แบรนด์ใน spec ใช้ชื่อ snake_case เช่น country_of_origin จึงไม่ต้องแปลงเป็น Product DTO

> ค่าที่แทน ? ส่งผ่าน array เช่น [id] ไม่ใช้การต่อ req.params.id เข้า SQL string โดยตรง

[S1–S4] §2.2; [R2] mysql2 Promise; โค้ดแยกชั้นเป็นคำตอบตัวอย่าง

เฉลย F1 / Product Repository: อ่านข้อมูล

src/repositories/product-repo.js — ส่วนที่ 1/2

```
import db from '../configs/db-config.js';

export async function findAll({ limit, offset }) {
  const [counts] = await db.query('select count(*) as total from products');
  const [rows] = await db.query(
    'select * from products order by id limit ? offset ?',
    [limit, offset]
  );
  return { rows, total: Number(counts[0].total) };
}

export async function findOne(id) {
  const [rows] = await db.query(`
    select p.*, b.name as brand_name
    from products p
    left join brands b on b.id = p.brand_id
    where p.id = ?`, [id]);
  return rows[0];
}

export async function findBySku(sku) {
  const [rows] = await db.query(
    'select id from products where sku = ?', [sku]
  );
  return rows[0];
}
```

นับ total แยกจากจำนวนแถวในหน้า

findAll เรียก SQL สองครั้ง: count(*) นับสินค้าในระบบ แล้ว select ... limit ... offset ดึงเฉพาะหน้าที่ต้องส่งกลับ จึงห้ามใช้ rows.length แทน total เพราะมันเป็นจำนวนของหน้าปัจจุบันเท่านั้น

```
const [counts] = await db.query('select count(*) as total from products');
const total = Number(counts[0].total);
```

JOIN เพื่อเอาชื่อแบรนด์

ใน products มี brand_id แต่ Detail DTO ต้องการ brandName ด้วย จึงเชื่อม products p กับ brands b ด้วย p.brand_id = b.id แล้วตั้งชื่อ b.name as brand_name เพื่อให้ mapper อ่านตรงกัน

products	brands	หลัง JOIN
id = 1, brand_id = 2	id = 2, name = Samsung	brand_name = Samsung

left join คงสินค้าไว้แม้ไม่มีแถวแบรนด์ที่ตรงกัน ส่วน inner join จะไม่คืนสินค้านั้น. ถ้ามี foreign key ที่ถูกต้อง ความแตกต่างนี้มักไม่เกิดในข้อมูลปกติ แต่ต้องรู้ตอนดัดแปลงโจทย์

> ต่อไฟล์นี้ด้วย F2 หน้าถัดไป ไม่ต้องสร้างไฟล์ใหม่ และไม่ใช้ productCode/productName/MSRP จาก classicmodels เดิม

[S1–S4] §2.3(1)–(2); [T1] product-repo.js; [R5] SELECT/JOIN; สมมติ schema ตามหน้า 4

เฉลย F2 / Product Repository: เขียนข้อมูล

src/repositories/product-repo.js — ส่วนที่ 2/2

```
// Only pass field names selected by the service allowlist.
export async function create(data) {
  const fields = Object.keys(data);
  const marks = fields.map(() => '?').join(', ');
  const [result] = await db.query(
    `insert into products (${fields.join(', ')}) values (${marks})`,
    Object.values(data)
  );
  return result.insertId;
}

export async function update(id, data) {
  const assignments = Object.keys(data)
    .map(field => `${field} = ?`).join(', ');
  await db.query(
    `update products set ${assignments} where id = ?`,
    [...Object.values(data), id]
  );
}

export async function remove(id) {
  const [result] = await db.query(
    'delete from products where id = ?', [id]
  );
  return result.affectedRows;
}
```

อ่านโค้ด dynamic SQL ให้เป็น

ถ้า data = {price: 25900, stock_quantity: 0} จะได้ Object.keys(data) เป็น ["price", "stock_quantity"] และ Object.values(data) เป็น [25900, 0]

```
update products set price = ?, stock_quantity = ? where id = ?
// values -> [25900, 0, id]
```

ชื่อคอลัมน์มาจาก allowlist ใน Service เท่านั้น ไม่รับชื่อคอลัมน์จากผู้ใช้โดยตรง ส่วนค่าต่าง ๆ ส่งเป็น placeholders. INSERT คืน result.insertId เพื่ออ่านสินค้าที่เพิ่งเพิ่มกลับไปทำ Detail DTO

DELETE คืน affectedRows. ถ้า 0 หมายถึงไม่มีแถวถูกลบตาม id; Service จึงตอบ 404. ส่วน update ไม่เอาจำนวนแถวที่เปลี่ยนจริงมาตัดสิน 404 เพราะส่งค่าเดิมก็ยังเป็นการแก้ไข resource ที่มีอยู่ได้

> อย่าเรียก create/update ของ repository นี้ด้วย object ดิบจาก req.body โดยข้าม Service เพราะจะข้ามรายการ field ที่อนุญาต

[S1–S4] §2.3(3)–(5); โค้ด SQL เป็นคำตอบตัวอย่างบน schema ที่สมมติ

เฉลย G1 / Product Service: อ่าน + คัด field

src/services/product-service.js — ส่วนที่ 1/2

```
import * as repo from '../repositories/product-repo.js';
import Mapper from '../mappers/product-mapper.js';
import { AppError } from '../middlewares/error-handler.js';

const allowedFields = [
  'sku', 'name', 'brand_id', 'category', 'price',
  'ram_gb', 'storage_gb', 'color', 'stock_quantity',
  'status', 'release_date'
];

function pickData(body) {
  if (!body || typeof body !== 'object' || Array.isArray(body)) {
    throw new AppError('JSON object required', 400);
  }
  const data = {};
  for (const key of allowedFields) {
    if (Object.hasOwn(body, key)) data[key] = body[key];
  }
  return data;
}

export async function getAllObjects(pagination) {
  const { rows, total } = await repo.findAll(pagination);
  const { page, limit } = pagination;
  return {
    data: rows.map(p => Mapper.toSummaryDTO(p)),
    pagination: {
      total, page, limit,
      totalPages: Math.ceil(total / limit),
      hasNextPage: page * limit < total,
      hasPrevPage: page > 1
    }
  };
}

export async function getObject(id) {
  const product = await repo.findOne(id);
  if (!product) throw new AppError('Product not found', 404);
  return Mapper.toDetailDTO(product);
}
```

pickData: รับเฉพาะ JSON object แล้วเลือกคอลัมน์ 11 ตัวที่อนุญาต; id และ brand_name ที่ผู้ใช้ส่งมาจะไม่ถูกนำไปเขียนฐานข้อมูล. Object.hasOwn ตรวจว่ามี key จริง จึงไม่ทำค่า 0 หาย

getAllObjects: ส่ง rows แต่ละตัวผ่าน Summary DTO แล้วประกอบ pagination. คืน {data, pagination} ให้ route ไม่แนบ totalItems เป็น property พิเศษของ array แบบ template เดิม

getObject: ไม่พบสินค้าให้ throw 404 ก่อนแปลง DTO. ถ้าพบจึงคืน Detail DTO ไม่ใช่ array ของสินค้า 1 ตัว

[S1–S4] §2.3(1)–(2); [T1] รูปแบบเดิมใน service/mapper; โค้ดเลือก field เป็นส่วนออกแบบของเฉลย

เฉลย G2 / Product Service: POST, PUT, DELETE

src/services/product-service.js — ส่วนที่ 2/2

```
export async function createObject(body) {
  const data = pickData(body);
  // Study policy; the spec does not list all required fields.
  for (const key of ['sku', 'name', 'brand_id', 'category', 'price']) {
    if (data[key] === undefined || data[key] === null) {
      throw new AppError(`Missing ${key}`, 400);
    }
  }
  if (await repo.findBySku(data.sku)) {
    throw new AppError('SKU already exists', 409);
  }
  const id = await repo.create(data);
  return getObject(id);
}

export async function updateObject(id, body) {
  await getObject(id);
  const data = pickData(body);
  if (Object.keys(data).length === 0) {
    throw new AppError('No fields to update', 400);
  }
  if (data.sku !== undefined) {
    const duplicate = await repo.findBySku(data.sku);
    if (duplicate && String(duplicate.id) !== String(id)) {
      throw new AppError('SKU already exists', 409);
    }
  }
  await repo.update(id, data);
  return getObject(id);
}

export async function removeObject(id) {
  const affected = await repo.remove(id);
  if (affected === 0) throw new AppError('Product not found', 404);
}
```

POST: เฉลยกำหนด 5 field ขั้นต้นว่าต้องมีเพื่อฝึก แต่ spec ไม่ได้ระบุ validation ครบ ต้องปรับตาม DDL/คำสั่งจริง. เช็ก sku ก่อนเพิ่มแล้วอ่านด้วย id ที่ MySQL สร้าง

PUT: ตรวจว่า id มีจริงก่อน จากนั้นเลือกเฉพาะ field ที่ส่งมา. เมื่อเปลี่ยน sku ต้องยอมให้ใช้ sku ของตัวเอง แต่ปฏิเสธ sku ที่อยู่กับ id อื่น. ค่าเดิมหรือ stock_quantity: 0 ต้องไม่หาย

DELETE: ลบแล้วดูจำนวนแถว. ต้องมี UNIQUE ที่ sku ในฐานข้อมูลเพื่อป้องกัน POST พร้อมกัน; precheck เพียงอย่างเดียวกัน race condition ไม่ได้ และ error handler รองรับ ER_DUP_ENTRY ไว้แล้ว

[S1–S4] §2.3(3)–(5); required fields/partial PUT/รูปแบบ error เป็นนโยบายเฉลย ไม่ใช่ข้อกำหนดที่เอกสารแจกแจง

เฉลย H / Product Route และการไหลของข้อมูล

src/routes/product-route.js

```
import express from 'express';
import * as service from '../services/product-service.js';
import { apiResponse } from '../mappers/api-response.js';
import { queryParser } from '../middlewares/pagination-middleware.js';

const router = express.Router();

router.get('/', queryParser, async (req, res) => {
  const result = await service.getAllObjects(req.pagination);
  return apiResponse(res, result.data, result.pagination);
});

router.get('/:id', async (req, res) => {
  return apiResponse(res, await service.getObject(req.params.id));
});

router.post('/', async (req, res) => {
  const data = await service.createObject(req.body);
  return apiResponse(res, data, null, 201);
});

router.put('/:id', async (req, res) => {
  const data = await service.updateObject(req.params.id, req.body);
  return apiResponse(res, data);
});

router.delete('/:id', async (req, res) => {
  await service.removeObject(req.params.id);
  return res.status(204).end();
});

export default router;
```

ลองไล่ GET /products?page=2&limit=10

queryParser สร้าง {page: 2, limit: 10, offset: 10} → Service ส่งไป Repository → SQL count + limit/offset → Mapper ตัดเหลือ 7 field → apiResponse ส่ง 200 พร้อม pagination

ลองไล่ POST /products

express.json อ่าน body → Service คัด field และเช็ก SKU → Repository INSERT → รับ insertId → query รายละเอียดพร้อม JOIN → Detail DTO → apiResponse(..., null, 201)

ทำไม DELETE ต่างจากข้ออื่น?

await removeObject ต้องเสร็จก่อนจึงตอบ 204. ถ้าไม่พบ ID จะ throw 404 และไม่ถึงบรรทัด end(). ห้ามส่ง {success: true}, null หรือข้อความใดตามหลัง 204

> นี่คือไฟล์สุดท้ายของเฉลยหลัก เมื่อติดตั้ง dependencies และมีฐานข้อมูลตรง schema แล้วจึงรัน npm start; สถานะ “Server running” อย่างเดียวไม่ได้พิสูจน์ว่า MySQL เชื่อมสำเร็จ

[S1–S4] §2.3; [R1, R2] async route และ Promise; โค้ดประกอบจากไฟล์ A–G

ทดสอบ 1 / GET, POST และ SKU ซ้ำ

GET ผ่าน Browser หรือ Terminal

```
curl.exe -i http://localhost:3000/api/67130500002/student-info
curl.exe -i http://localhost:3000/api/67130500002/brands
curl.exe -i http://localhost:3000/api/67130500002/brands/1
curl.exe -i "http://localhost:3000/api/67130500002/products?page=2&limit=10"
curl.exe -i http://localhost:3000/api/67130500002/products/1
```

Windows ใช้ curl.exe เพื่อไม่สับสนกับ alias ใน PowerShell; Bash/macOS ใช้ curl. -i แสดง HTTP status/header ด้วย ต้องเปลี่ยนรหัสใน URL ให้ตรงกับ app.js

POST ด้วยไฟล์ JSON เพื่อลดปัญหาเครื่องหมาย quote

`practice/product-new.json` — เปลี่ยน SKU ได้ก่อนทดสอบ

```
{
  "sku": "STUDY-SKU-999",
  "name": "Study Phone 2026",
  "brand_id": 1,
  "category": "FLAGSHIP",
  "price": 29900,
  "ram_gb": 8,
  "storage_gb": 256,
  "color": "Black",
  "stock_quantity": 10,
  "status": "AVAILABLE",
  "release_date": "2026-01-01"
}
```

```
curl.exe -i -X POST http://localhost:3000/api/67130500002/products -H "Content-Type: application/json"
--data-binary "@practice/product-new.json"
```

ส่งครั้งแรกควรได้ 201 เมื่อ brand_id มีจริงและ schema รับข้อมูลนี้ได้. จด data.id ของสินค้าที่สร้าง เช่นได้ 76 ก็ใช้ 76 ใน PUT/DELETE ห้ามสมมติว่า id จะเป็น 76 ทุกครั้ง

ส่ง POST เดิมซ้ำอีกครั้งควรได้ 409 เพราะ sku เดิม. อย่าเปลี่ยน sku ก่อนทดสอบกรณีนี้ และอย่าทดสอบการลบกับข้อมูล seed ที่อยากเก็บไว้

> JSON request ใช้ snake_case เช่น brand_id แต่ response DTO ใช้ brandId. การส่งกลับ DTO ไม่ได้แปลว่า API รับ input เป็น camelCase ด้วย

[S1–S4] §2.3(3); คำสั่งและข้อมูล STUDY-SKU-999 เป็นตัวอย่างทดสอบที่สร้างเพิ่ม

ทดสอบ 2 / PUT, DELETE และผลลัพธ์ที่ควรได้

แก้สินค้าที่เพิ่งสร้าง ไม่ใช้ id แบบเดาสุ่ม

practice/product-update.json

```
{
  "price": 25900,
  "stock_quantity": 0
}
```

```
curl.exe -i -X PUT http://localhost:3000/api/67130500002/products/76 -H "Content-Type: application/
json" --data-binary "@practice/product-update.json"
curl.exe -i http://localhost:3000/api/67130500002/products/76
curl.exe -i -X DELETE http://localhost:3000/api/67130500002/products/76
curl.exe -i http://localhost:3000/api/67130500002/products/76
```

76 เป็นเพียงตัวอย่าง เปลี่ยนเป็น id ที่ได้จาก POST. ตรวจ PUT ว่า price = 25900, stockQuantity = 0. DELETE ต้อง 204 และ body ว่าง จากนั้น GET ต้อง 404

กรณีตรวจ	ผลตาม spec / ตามเฉลย
GET student-info	200; ชื่อ รหัส key ตรง spec
GET brands รายการ / แบรนด์ที่มี	200; array / object
GET brand หรือ product ที่ไม่มี	404
GET products ไม่ส่ง page	200; page เริ่มที่ 1 ตาม spec
GET products ส่ง limit=3	data ไม่เกิน 3; total เป็นทั้งหมด
Summary DTO	มีเพียง 7 field ที่ระบุ
Detail DTO	มี brandName และรายละเอียดครบ
POST SKU ใหม่ / ซ้ำ	201 / 409
PUT ID ที่มี / ไม่มี	200 / 404
PUT ค่าเดิม / จำนวนสต็อกเป็น 0	200 และค่าไม่หาย ตามเฉลย
DELETE ที่มี / ลบซ้ำ	204 ไม่มี body / 404
page=abc / limit=0 / body ว่าง	400 ตามนโยบายของเฉลย ไม่ใช่ spec ที่แจกแจงไว้

ตรวจ JSON ด้วยตาให้ถูกจุด

ไม่ควรมี pagination ใน student-info หรือ detail. total/page/limit/totalPages เป็นตัวเลข ไม่ใช่ string. hasNextPage/ hasPrevPage เป็น boolean. ไม่ต้องทำให้ JSON มีทศนิยม .00 ตลอด เพราะ 29900 และ 29900.00 เป็น JSON Number ค่าเดียวกัน

[S1–S4] §2–3; กรณีที่ระบุ “ตามเฉลย” เป็นการทดสอบเพิ่มเติม

แก้ Error / อ่านอาการแล้วหาไฟล์ให้ถูก

อาการ	สาเหตุที่ต้องตรวจ	จุดแก้
Missing script: start	ไม่มี start ใน package.json	เพิ่ม node src/app.js ใน scripts
Cannot find package express/mysql2	ยังไม่ได้ติดตั้งหรือเปิดผิดโฟลเดอร์	npm install ที่โฟลเดอร์ package.json
ERR_MODULE_NOT_FOUND	path/import หรือตัวพิมพ์ผิด	ใช้ชื่อไฟล์ตรงจริง และ .js ใน relative import
does not provide an export named ...	named/default export ไม่ตรงกัน	เช็ก export และ import คู่กัน
ECONNREFUSED ที่ MySQL	MySQL ไม่ทำงานหรือ host/port ผิด	ตรวจ service ของ MySQL และ db-config
ER_ACCESS_DENIED_ERROR	user/password หรือสิทธิ์ไม่ถูก	db-config หรือ environment
ER_BAD_DB_ERROR	ยังไม่มี database ชื่อนั้น	นำเข้า SQL และตรวจ DB_NAME
ER_NO_SUCH_TABLE	ฐานข้อมูล/ชื่อตารางผิด	show tables; ตรวจ schema
ER_BAD_FIELD_ERROR	ยังใช้ productCode/MSRP หรือ column ผิด	describe products; แก้ repository/ mapper
ER_BAD_NULL_ERROR / NO_DEFAULT...	คอลัมน์บังคับไม่ได้รับค่า	ตรวจ schema + request body
ER_DUP_ENTRY	ชน unique เช่น SKU	409; อย่าแก้ให้บันทึกซ้ำได้
ER_NO_REFERENCED_ROW_2	brand_id ไม่พบใน brands	ใช้ brand ที่มีจริง ไม่ปิด foreign key
EADDRINUSE	มี server ฟัง port 3000 อยู่	หยุดเฉพาะ server เก่าที่ตนเปิดด้วย Ctrl+C
Cannot GET /...	path / prefix / method ไม่ตรง	เทียบ app.js + router + URL
req.body เป็น undefined	ไม่มี express.json หรือ Content-Type ผิด	middleware ก่อน route; raw JSON
headers already sent	ส่ง response ซ้ำหรือทำงานต่อหลังส่ง	ใช้ return และไม่เรียก res ซ้ำ
400 หลังส่ง JSON	JSON ผิด เช่น comma เกิน หรือ single quotes	ใช้ double quotes ใน JSON; ดู body
แก้ code แล้วไม่เปลี่ยน	npm start ไม่ได้ auto reload	Ctrl+C แล้ว npm start ใหม่

จุดผิดใน template เดิมที่เกี่ยวกับโจทย์นี้

app.js ใช้ ID ตัวอย่างเก่าและเปิด offices/productlines; db-config ใช้ classicmodels; product-repo ใช้ productCode และ create อ่าน productLine; product-mapper ใช้ productName/MSRP; pagination เดิมไม่สร้างค่า default เมื่อ query ขาด. จึงเปลี่ยนเฉพาะรหัสนักศึกษาอย่างเดียวไม่พอ

> การเห็น 500 ให้เริ่มอ่าน Terminal ของ Server ก่อนแก้ URL หรือสุ่มเปลี่ยน SQL ไม่ปิด constraints เพียงเพื่อให้ error หาย

[T1] ตรวจโค้ด template โดยตรง; [R1, R4] Express/ESM; ตาราง error เป็นคู่มือแก้ปัญหาเพิ่มเติม

Self-test, ฐานข้อมูลฝึก และการส่งงาน

แยกการทดสอบสองชนิด

ชุดตรวจของผู้สอน	ชุดฝึกที่แนบกับชีท
node tests/test-runner.mjs รหัส	npm run test:study
ไม่มีอยู่ใน ZIP ต้นฉบับที่แนบมา	อยู่ใน study-tests/ ของชุดโค้ดนี้
ตรวจคะแนนจริงตาม implementation ผู้สอน	ทดสอบตรรกะด้วย mysql2 จำลอง
ต้องรันกับ Express + ฐานข้อมูลจริงตามโจทย์	ไม่ยืนยัน SQL บน MySQL หรือ HTTP จริง

ผลที่ตรวจในงานนี้: source JavaScript ทั้ง 12 ไฟล์ผ่าน node --check; ชุดฝึก 24 กรณีย่อยผ่านบน Node.js 22.16.0 (TAP นับรวมชุดครอบเป็น 25). ไม่ได้รัน official test runner และไม่อ้างว่าผ่าน MySQL จริง

ฝึกได้แม้ยังไม่มี SQL ข้อสอบ

Command Prompt / Bash — ทำในฐานข้อมูลฝึกเท่านั้น

```
mysql -u root -p < practice/practice-only.sql
```

PowerShell

```
$env:DB_NAME = 'mobile_distributor_practice'
$env:DB_PASSWORD = 'YOUR_MYSQL_PASSWORD'
npm start
```

practice-only.sql เป็น schema/seed ที่เขียนใหม่เพื่อฝึก ไม่ใช่ไฟล์ที่หายไปจากข้อสอบ มีเพียงสินค้า 3 รายการและ 2 แบรนด์ ไม่ใช่ 75 รายการตาม JSON ตัวอย่าง. seed ให้รันครั้งเดียวในฐานข้อมูลฝึกที่ยังว่าง

> กลับไปทำข้อสอบจริงต้องเปลี่ยน DB_NAME กลับเป็น mobile_distributor และใช้ SQL ของผู้สอน ห้ามส่งฐานข้อมูลฝึกแทนของจริง

บีบอัดก่อนส่ง

สำรองโฟลเดอร์งานแล้วลบ node_modules เฉพาะในสำเนาสำหรับส่ง ไม่ลบ package.json, package-lock.json, src หรือไฟล์ SQL/tests ที่ผู้สอนกำหนดให้ส่ง. ไม่ต้องแนบ .idea, __MACOSX, .DS_Store หรือไฟล์ฝึกส่วนตัวถ้าไม่ได้ร้องขอ

```
67130500002_exam.zip
67130500003_exam.zip
67130500004_exam.zip
```

เลือกชื่อให้ตรงรหัสของตนเอง และตรวจเปิด ZIP ว่ามี source จริง ไม่ใช่ shortcut หรือ ZIP ซ้อนผิดโฟลเดอร์

[S1–S4] §3–4; ผล test และฐานข้อมูลฝึกเป็นงานตรวจ/ตัวอย่างที่สร้างในชีทนี้

แนวโจทย์อื่น / อ่านแบบไม่เดาคะแนน

ส่วนต่อจากนี้คือแนวฝึกเพิ่ม ไม่ใช่ข้อกำหนดของ spec ทั้ง 4 ไฟล์ ไม่มีหลักฐานพอจะให้เปอร์เซ็นต์ว่าอาจารย์จะออกอะไร จึงจัดลำดับจากความใกล้กับทักษะในโจทย์ ไม่ใช่ความน่าจะเป็นทางสถิติ

แนวฝึก	จุดที่เปลี่ยน	เหตุผลที่ควรฝึก
เปลี่ยน resource เป็น students/books	ชื่อตาราง, primary key, field, DTO	โครง CRUD เดิมใช้ได้
ค้นหาด้วย keyword	req.query.q + where ... like ?	ต่อยอด GET รายการโดยตรง
กรอง brand/status/category	WHERE แบบมีเงื่อนไข	ต้องทำให้ count กับ data ตรงกัน
ช่วงราคาและเรียงลำดับ	minPrice/maxPrice + order by	ฝึก validation และ allowlist
/brands/:id/products	parent id จาก params	ฝึกความสัมพันธ์ตาราง
GET /products/stats	count/sum/avg/group by	ฝึก aggregate + route ordering
PATCH สต็อก	update ค่าเดียว + ตรวจจำนวน	ต่อยอดระบบสินค้าและสต็อก
Validation รายละเอียด	ข้อมูลขาด/ผิดชนิด/ติดลบ	ต่อยอด 400/404/409
PUT แบบแทนที่ทั้ง resource	require ทุก field ที่โจทย์กำหนด	แยกจาก partial update
Transaction / orders	หลาย SQL ต้องสำเร็จด้วยกัน	แนวระดับสูงกว่า ไม่อยู่ใน spec
Login/JWT/upload	เพิ่มระบบอื่นจำนวนมาก	ไม่มีหลักฐานในไฟล์ให้จัดเป็นเรื่องหลัก

วิธีแปลงเฉลยไปเป็นโจทย์ใหม่

```
1. Read endpoint + method + request + response + status
2. Inspect real SQL columns and constraints
3. Rename table / primary key in repository
4. Change allowedFields and validation in service
5. Change DTO keys and route base path
6. Test success, not-found, duplicate, invalid data
```

อย่าใช้ findBySku กับระบบนักศึกษาโดยไม่เปลี่ยนเป็น key ที่โจทย์ระบุ และอย่าลืมเปลี่ยน brandName JOIN เมื่อโจทย์ใหม่เป็นความสัมพันธ์คนละแบบ

แนวฝึกที่ผู้จัดทำเสนอจากทักษะใน [S1–S4] ไม่ใช่รายละเอียดข้อสอบเพิ่มเติมที่พบในไฟล์

แนวฝึก 1A / ค้นหา กรองราคา เรียงข้อมูล

สร้าง src/repositories/product-filter.js ด้วยโค้ดนี้ก่อน แล้วเชื่อมกับ 3 ชั้นตามหน้าถัดไป. ตัวอย่างนี้เป็นส่วนเสริม ไม่เปิดใช้ในเฉลยหลักโดยอัตโนมัติ

src/repositories/product-filter.js — ไฟล์ใหม่สำหรับแนวฝึก

```
import { AppError } from '../middlewares/error-handler.js';

export function buildFilter(q) {
  const conditions = [];
  const values = [];
  if (q.brandId !== undefined) {
    const brandId = Number(q.brandId);
    if (!Number.isSafeInteger(brandId) || brandId < 1) {
      throw new AppError('Invalid brandId', 400);
    }
    conditions.push('p.brand_id = ?');
    values.push(brandId);
  }
  if (q.status !== undefined) {
    if (typeof q.status !== 'string') {
      throw new AppError('Invalid status', 400);
    }
    conditions.push('p.status = ?');
    values.push(q.status);
  }
  if (q.q !== undefined) {
    if (typeof q.q !== 'string') throw new AppError('Invalid q', 400);
    conditions.push('(p.name like ? or p.sku like ?)');
    values.push(`%${q.q}%`, `%${q.q}%`);
  }
  for (const [key, op] of [['minPrice', '>='], ['maxPrice', '<=']]) {
    if (q[key] === undefined) continue;
    const number = Number(q[key]);
    if (!Number.isFinite(number) || number < 0 || q[key] === '') {
      throw new AppError(`Invalid ${key}`, 400);
    }
    conditions.push(`p.price ${op} ?`);
    values.push(number);
  }
  if (q.minPrice !== undefined && q.maxPrice !== undefined &&
      Number(q.minPrice) > Number(q.maxPrice)) {
    throw new AppError('minPrice exceeds maxPrice', 400);
  }
  const columns = { id: 'p.id', price: 'p.price', name: 'p.name' };
  const sort = q.sort ?? 'id';
  if (typeof sort !== 'string' || !Object.hasOwn(columns, sort)) {
    throw new AppError('Invalid sort', 400);
  }
  const order = String(q.order ?? 'asc').toLowerCase();
  if (!['asc', 'desc'].includes(order)) {
    throw new AppError('Invalid order', 400);
  }
  return {
    where: conditions.length ? 'where ' + conditions.join(' and ') : '',
    values,
    orderBy: `${columns[sort]} ${order}, p.id asc`
  };
}
```

ชื่อคอลัมน์และ asc/desc ต้องมาจากรายการที่อนุญาต ไม่ใส่ req.query.sort ลง SQL โดยตรง. สำหรับ q ตัวอย่างนี้ให้ % และ _ มีความหมาย wildcard ของ LIKE; ถ้าโจทย์ต้องค้นหาอักขระเหล่านี้แบบตรงตัวต้องเพิ่มการ escape

แนวฝึกเพิ่มเติม; [R5] WHERE/ORDER BY/LIKE; โค้ดเขียนขึ้นสำหรับตัวอย่าง

แนวฝึก 1B / เชื่อม Filter โดยไม่ทำ Pagination พัง

แก้ src/repositories/product-repo.js

```
// Add near the top of product-repo.js:
import { buildFilter } from './product-filter.js';

// Replace only findAll; keep other repository functions.
export async function findAll({ limit, offset }, query = {}) {
  const f = buildFilter(query);
  const [counts] = await db.query(
    `select count(*) as total from products p ${f.where}`,
    f.values
  );
  const [rows] = await db.query(
    `select p.* from products p ${f.where}
     order by ${f.orderBy} limit ? offset ?`,
    [...f.values, limit, offset]
  );
  return { rows, total: Number(counts[0].total) };
}
```

แก้ Service และ Route เพิ่มอีกสองจุด

Service — เปลี่ยน signature และบรรทัดเรียก repo เท่านั้น

```
export async function getAllObjects(pagination, query = {}) {
  const { rows, total } = await repo.findAll(pagination, query);
  // Keep the remaining DTO/pagination code from G1 unchanged.
}
```

Route — ภายใน GET /products เดิม

```
const result = await service.getAllObjects(
  req.pagination, req.query
);
```

วงเล็บปิดในตัวอย่าง Service เป็นโครงย่อเพื่อแสดงจุดแก้ ไม่ใช่แทนทั้งฟังก์ชันแล้วลบ return เดิม; ให้เก็บการสร้าง data/pagination จาก G1 ไว้ทั้งหมด

ทดสอบกับข้อมูลที่รู้ผลล่วงหน้า

```
/products?q=Phone&brandId=1&page=1&limit=2
/products?minPrice=10000&maxPrice=30000&sort=price&order=desc
```

หากค้นหาแล้วเหลือ 12 รายการ limit = 5 ต้องได้ total = 12 และ totalPages = 3 ไม่ใช่ total ของฐานข้อมูลทั้งหมด. WHERE ของ count และ query รายการต้องเหมือนกันและใช้ค่าตามลำดับเดียวกัน

> **กับดัก:** ถ้า query ข้อมูลใช้ brand_id = 1 แต่ count ไม่กรอง brand_id จำนวนหน้าและ hasNextPage จะผิด ถึงแม้รายการที่แสดงจะดูถูกแล้วก็ตาม

ดัดแปลงเป็น /brands/:id/products

ใน brand route ตรวจ parent ด้วย brandService.getObject(req.params.id) ก่อน แล้วเรียก productService.getAllObjects(req.pagination, {brandId: req.params.id}). แบรนด์ไม่มี → 404; แบรนด์มีแต่ยังไม่มีสินค้า → 200 พร้อม data: [] ตามนโยบายตัวอย่าง

แนวฝึกเพิ่มเติม; สูตร pagination อิงเฉลย G1; ไม่ใช่ endpoint ที่เพิ่มมาใน spec

แนวฝึก 2 / ลดสต็อกโดยไม่ให้ติดลบ

โจทย์สมมติ: PATCH /products/:id/stock รับ {"quantity": 3} หมายถึงขายออก 3 ชิ้น ต้องไม่ให้สต็อกติดลบ ตัวอย่างแสดง SQL ใน route เพื่อเห็น logic รวมกัน; เมื่อนำไปจัดชั้นให้ย้าย SQL เข้า repository

```
// In product-route.js, also import db and AppError:
// import db from '../configs/db-config.js';
// import { AppError } from '../middlewares/error-handler.js';
router.patch('/:id/stock', async (req, res) => {
  const id = Number(req.params.id);
  const quantity = req.body?.quantity;
  if (!Number.isSafeInteger(id) || id < 1 ||
      !Number.isSafeInteger(quantity) || quantity < 1) {
    throw new AppError('Positive id and quantity required', 400);
  }
  const [result] = await db.query(`
    update products
    set stock_quantity = stock_quantity - ?
    where id = ? and stock_quantity >= ?`,
    [quantity, id, quantity]);

  if (result.affectedRows === 0) {
    const [rows] = await db.query(
      'select id from products where id = ?', [id]
    );
    if (rows.length === 0) throw new AppError('Product not found', 404);
    throw new AppError('Insufficient stock', 409);
  }
  return apiResponse(res, await service.getObject(id));
});
```

ทำไมไม่อ่านจำนวนเก่ามาลบใน JavaScript?

ถ้ามีสองคำขอพร้อมกัน ทั้งคู่สามารถอ่านจำนวนเก่าเดียวกันก่อนเขียนกลับได้. การใช้ update ... stock_quantity >= ? ทำให้เงื่อนไขตรวจจำนวนและการลดอยู่ในคำสั่งเดียว ไม่ใช้จำนวนเก่าที่อ่านก่อนหน้า

ตัวอย่าง	ผลที่แนวฝึกกำหนด
stock = 5, quantity = 3	สำเร็จ เหลือ 2
stock = 2, quantity = 3	409 สต็อกไม่พอ
ไม่มี product id	404
quantity = 0, -1, 1.5 หรือ "3"	400 เพราะกำหนดให้เป็น JSON integer บวก

การเช็ก existence หลัง update ที่ไม่สำเร็จใช้เพื่อจำแนก 404/409 ในตัวอย่าง. ถ้ามีการลบ/เปลี่ยนข้อมูลพร้อมกันและต้องการผลที่ผูกกันแบบเคร่งครัด ให้ใช้ transaction/locking ตามหน้าถัด ๆ ไป

แนวฝึกเพิ่มเติม ไม่อยู่ใน spec; [R6] MySQL transaction/locking; policy 400/409 ของโจทย์สมมติ

แนวฝึก 3 / Stats, GROUP BY และลำดับ Route

โจทย์สมมติ: GET /products/stats แสดงจำนวนชนิดสินค้า จำนวนชิ้นรวม และมูลค่าสต็อก ไม่ใช่เฉพาะจำนวนสินค้าที่แสดงในหน้าปัจจุบัน

```
// Add before router.get('/:id', ...) in product-route.js.
// Also import db from '../configs/db-config.js';
router.get('/stats', async (req, res) => {
  const [rows] = await db.query(`
    select count(*) as product_count,
           coalesce(sum(stock_quantity), 0) as total_stock,
           coalesce(sum(price * stock_quantity), 0) as stock_value
    from products`);
  const r = rows[0];
  return apiResponse(res, {
    productCount: Number(r.product_count),
    totalStock: Number(r.total_stock),
    stockValue: Number(r.stock_value)
  });
});
```

วาง /stats ก่อน /:id: ไม่เช่นนั้นคำว่า stats อาจถูก route แบบ :id รับไปเป็น id และทำให้ผลตอบผิด endpoint

นับจำนวนสินค้าต่อแบรนด์ รวมแบรนด์ที่ยังไม่มีสินค้า

```
select b.id, b.name, count(p.id) as product_count
from brands b
left join products p on p.brand_id = b.id
group by b.id, b.name
order by b.id;
```

ใช้ count(p.id) ไม่ใช่ count(*): แบรนด์ที่ไม่มีสินค้ายังมีแถวจาก left join แต่ p.id เป็น null จึงต้องนับเฉพาะ p.id เพื่อให้ได้ 0 ไม่ใช่ 1

มูลค่าสต็อกต่างจากยอดขาย

sum(price * stock_quantity) คือมูลค่าของสินค้าที่มีอยู่ ไม่ใช่ยอดขายสะสม. ถ้าโจทย์ถามรายได้จาก orders ต้องอ่านตารางคำสั่งซื้อจริง ไม่สามารถแทนด้วยราคาสต็อกได้

> aggregate บางชนิดอาจกลับมาเป็น string หรือ null ตามชนิด SQL และกรณีไม่มีข้อมูล จึงออกแบบ DTO ให้ชัดเจน; coalesce(..., 0) ใช้แทนค่า null ของผลรวม

แนวฝึกเพิ่มเติม; [R5] SELECT / GROUP BY; โค้ดและนิยามตัวชี้วัดของโจทย์สมมติ

แนวฝึก 4 / Validation, PUT และ PATCH

Validation ต้องทำตามเงื่อนไขที่โจทย์ใหม่ระบุ ไม่ควรเดาว่า category มีแค่ FLAGSHIP หรือ status มีแค่ AVAILABLE เพราะ spec ให้มาเพียงตัวอย่าง ไม่ใช่ enum ทั้งหมด

ตัวอย่างเพิ่มใน `product-service.js` แล้วเรียกหลัง `pickData`

```
function validateNumbers(data) {
  if (data.price !== undefined &&
      (typeof data.price !== 'number' ||
       !Number.isFinite(data.price) || data.price < 0)) {
    throw new AppError('price must be a non-negative number', 400);
  }
  for (const key of ['stock_quantity', 'ram_gb', 'storage_gb']) {
    if (data[key] === undefined) continue;
    if (!Number.isSafeInteger(data[key]) || data[key] < 0) {
      throw new AppError(`${key} must be a non-negative integer`, 400);
    }
  }
  if (data.brand_id !== undefined &&
      (!Number.isSafeInteger(data.brand_id) || data.brand_id < 1)) {
    throw new AppError('brand_id must be a positive integer', 400);
  }
}
```

นโยบายของตัวอย่างนี้ไม่รับ null หรือ string ใน field ตัวเลข ถ้า schema อนุญาต null สำหรับ ram_gb ต้องปรับกฎให้ตรง. price = 0 อนุญาตเฉพาะเมื่อโจทย์ใช้ non-negative; ถ้าโจทย์บอกต้องมากกว่า 0 ให้เปลี่ยน < 0 เป็น <= 0

เช็กข้อความว่างอย่างไร?

```
if (typeof data.sku !== 'string' || data.sku.trim() === '') {
  throw new AppError('sku must be non-empty text', 400);
}
```

ตรวจความยาวตาม varchar จริงด้วย และตรวจว่า input เป็น scalar ตามคอลัมน์ ไม่ปล่อย object/array เป็นค่า SQL โดยไม่ตั้งใจ. ข้อความวันที่ควรตรวจทั้งรูปแบบและวันที่มีอยู่จริง ไม่ใช่ regex อย่างเดียว

PUT ในมาตรฐาน กับ PUT ในตัวอย่างนี้

HTTP กำหนด PUT ในความหมายสร้าง/แทนที่ state ของ resource ตาม representation ที่ส่งมา ส่วน PATCH ใช้แก้ไขบางส่วน. แต่ spec นี้ระบุเพียง PUT “แก้ไขข้อมูลสินค้า” โดยไม่แจกแจง field บังคับ จึงบอกไม่ได้ว่าผู้ตรวจคาด full replacement หรือ partial update

เฉลยหลักเลือก partial update อย่างเปิดเผย. ถ้าโจทย์ใหม่กำหนด full replacement ต้อง require ทุก field ที่ต้องแทนที่ และตัดสินค่า field ที่ไม่ส่งมาตาม contract ไม่ใช้ COALESCE เพื่อคงค่าเดิมโดยอัตโนมัติ

แนวฝึกเพิ่มเติม; [R7] PUT; [R8] PATCH; validation/partial update เป็นทางเลือกที่ระบุ ไม่ใช่เงื่อนไขบังคับจาก spec

แนวฝึก 5 / Transaction เมื่อโจทย์มีหลายตาราง

ตัวอย่างระดับสูงกว่า spec: การขายหนึ่งครั้งต้องลดสต็อกและสร้าง order ให้สำเร็จพร้อมกัน ถ้าบันทึก order ไม่สำเร็จต้องไม่ปล่อยให้สต็อกถูกลดค้างอยู่

โครงสำหรับศึกษา ไม่ใช่โค้ดที่วางในเฉลยหลักแล้วรันได้ทันที

```
// Conceptual example: orders table must exist in the NEW exam schema.
const conn = await db.getConnection();
try {
  await conn.beginTransaction();
  const [rows] = await conn.query(
    'select id, stock_quantity from products where id = ? for update',
    [productId]
  );
  if (!rows[0]) throw new AppError('Product not found', 404);
  if (rows[0].stock_quantity < quantity) {
    throw new AppError('Insufficient stock', 409);
  }
  await conn.query(
    'update products set stock_quantity = stock_quantity - ? where id = ?',
    [quantity, productId]
  );
  await conn.query(
    'insert into orders (product_id, quantity) values (?, ?)',
    [productId, quantity]
  );
  await conn.commit();
} catch (err) {
  await conn.rollback();
  throw err;
} finally {
  conn.release();
}
```

> ตาราง orders และคอลัมน์ในตัวอย่างเป็นสมมติฐานใหม่ ไม่ได้ปรากฏในไฟล์ spec ที่แนบ ต้องมี DDL จริงก่อนใช้งาน และต้อง validate productId/quantity เป็นจำนวนเต็มบวกก่อนเข้า transaction

ประเด็นสำคัญ: ใช้ conn เดียวกันทุก query ตั้งแต่เริ่มจน commit/rollback. อย่าสลับไปใช้ db.query ของ pool กลางเพราะอาจได้คนละ connection. finally ต้อง release เพื่อคืน connection

SELECT ... FOR UPDATE ล็อกแถวใน transaction ของ InnoDB เพื่อไม่ให้คำขออื่นเปลี่ยนแถวนั้นระหว่างตรวจและแก้. การจัดการ deadlock/retry และรายละเอียด order หลายสินค้าเป็นเรื่องต่อยอด ไม่ใช่ requirement ของชุดข้อสอบนี้

> ไม่ควรรีบเพิ่ม transaction หรือ orders ลงข้อสอบเดิมที่ยังไม่ได้ร้องขอ ให้ทำ CRUD/DTO/pagination ตาม spec เดิมให้ครบก่อน

แนวฝึกเพิ่มเติม; [R2, R6] connection pool, transaction และ locking reads; ไม่มีตาราง orders ใน spec ที่ให้มา

โจทย์ลองทำ / พร้อมเฉลยและเหตุผล

โจทย์ฝึก	คำตอบและเหตุผล
มี 47 รายการ page=4 limit=6	offset=18; totalPages=8; hasNextPage=true; hasPrevPage=true
มี 47 รายการ page=8 limit=6	offset=42; หน้านี้มีได้ 5 รายการ; next=false; prev=true
GET /products/999 ไม่พบสินค้า	404 ไม่ใช่ 200 พร้อม data:null และไม่ใช่ 500
POST SKU ที่มีแล้ว	409; ตรวจ SKU + รองรับ unique constraint error
PUT /products/1 ส่ง {stock_quantity:0}	คงค่า 0 ไว้; Object.hasOwn/undefined ไม่ใช้ truthy
PUT ส่งราคาเท่าเดิมทุกประการ	200 เมื่อ resource มีจริง; ไม่ใช้ changedRows ตัดสิน existence
DELETE สำเร็จควรส่ง {success:true} หรือไม่	ไม่ส่ง body; ใช้ res.status(204).end()
GET รายการควรมี brandName หรือไม่	ตาม spec นี้ไม่ควร; Summary ระบุเฉพาะ 7 field
GET detail เอา brandName จากไหน	JOIN brands.name as brand_name แล้ว map เป็น brandName
limit=10 แต่ rows.length=5 และ total=75	rows.length บอกข้อมูลหน้านี้ ไม่ใช่ total ทั้งระบบ
req.query.page เป็น "2" ควรบวก 1 ทันทีไหม	ต้องแปลงก่อน ไม่เช่นนั้น "2" + 1 ได้ "21"
ให้ผู้ใช้ส่ง sort=price desc; drop...	ห้ามต่อเข้า SQL; ตรวจ sort/order ด้วย allowlist
ค้นหาแล้ว data ถูก แต่ totalPages มากเกิน	ตรวจ COUNT ว่าใช้ WHERE เดียวกับ query รายการหรือไม่
ไม่ส่ง page แต่ส่ง limit=5	ใช้ default page=1; offset=0; อย่าปิด pagination
product id ไม่มี แต่ PUT body ก็ไม่ถูก	เฉลยนี้ตรวจ existence ก่อน จึง 404; contract ใหม่อาจกำหนดลำดับอื่น
เลข 75 ใน JSON ตัวอย่างควรฝังในโค้ดไหม	ไม่ควร; เป็นตัวอย่างผล ไม่ใช่จำนวนจริงตลอดไป

ฝึกเปลี่ยน entity: books

สมมติ books มี id, isbn, title, price, author_id และ authors มี id, name. เปลี่ยน findBySku เป็น findByIsbn, เปลี่ยน allowlist และ Summary DTO ให้ตรงโจทย์; Detail ใช้ JOIN authors.name as author_name. อย่าเปลี่ยนแค่ URL แล้วปล่อย SQL เป็น products

> คำตอบข้อ validation และลำดับ error เป็นพฤติกรรมของเฉลย ไม่ควรอ้างเป็นเกณฑ์คะแนนของผู้สอนโดยไม่มี spec เพิ่มเติม

โจทย์ฝึกสร้างขึ้นจาก [S1–S4]; คำตอบเชิงคำนวณและการไล่โค้ดของเฉลย

คำถามแนวอธิบาย / ตอบให้ได้ก่อนเข้าห้อง

คำถาม	แนวคำตอบสั้นที่ควรเข้าใจ
Route / Service / Repository ต่างกันอย่างไร	Route รับ HTTP; Service ทำกฎงาน; Repository ติดต่อ DB; Mapper จัดรูปข้อมูลออก
DTO คืออะไร	object ที่เลือก/เปลี่ยนชื่อ field สำหรับส่งระหว่างระบบ ไม่จำเป็นต้องตรงทุก column ใน DB
ทำไมต้องมี await	รอ Promise ของ query เพื่อใช้ผล ไม่ใช่ใช้ Promise แทนข้อมูล; ไม่ได้แปลว่า process ทั้งหมดหยุดทำงาน
const [rows] หมายถึงอะไร	destructuring เอาผลสมาชิกแรกจาก [rows, fields] ของ mysql2 query
rows กับ rows[0] ต่างกันอย่างไร	rows คือ array หลายแถว; rows[0] คือแถวแรกหรือ undefined ถ้าไม่มี
insertId กับ affectedRows ต่างกันอย่างไร	insertId คือ key auto-increment ที่สร้าง; affectedRows คือจำนวนแถวที่คำสั่งกระทบตามผล DB
ทำไมเขียน ?	แยกค่าข้อมูลออกจาก SQL; ต้องส่งค่าตามลำดับ ไม่ใช้ ? เป็นชื่อคอลัมน์สำหรับ ORDER BY แบบทั่วไป
query กับ execute เหมือนกันไหม	mysql2 execute ใช้ prepared statement; query มีการจัดรูป/escape ฝั่ง client; อย่าสลับ API แบบไม่ดู parameter ที่ใช้
200 / 201 / 204 ต่างกันอย่างไร	สำเร็จทั่วไป / สร้าง resource ใหม่ / สำเร็จแต่ไม่มีเนื้อหาตอบกลับ
404 / 409 ต่างกันอย่างไร	ไม่พบ resource / คำขอขัดแย้งกับ state เช่น SKU ซ้ำ
middleware ทำงานเมื่อไร	ตามลำดับที่ลงทะเบียน; next() ส่งต่อ; error handler รับ error ด้วย 4 parameters
export default กับ export function	default import เลือกชื่อในไฟล์รับได้; named import ใช้ชื่อที่ export ในวงเล็บปีกกา
ทำไมชื่อ import ต้องมี .js	ESM relative import ของ Node ต้องระบุนามสกุลไฟล์
ทำไม DATE ไม่ควรแปลง UTC แบบไม่คิด	วันที่ปฏิทินไม่มีเวลา; การแปลง timezone อาจเลื่อนวันโดยไม่ต้องการ
ทำไม password ไม่ควรอยู่ใน repo จริง	เป็น credential ของระบบ; ตัวอย่างมี placeholder และรองรับ environment เพื่อแยกค่าออกจาก source
timestamp ใน response เหมือน created_at ไหม	ไม่เหมือน; timestamp คือเวลาตอบคำขอ ส่วน created_at คือเวลาสร้างข้อมูลตาม DB

กรณี Express 4 แทน 5

```
router.get('/', async (req, res, next) => {
  try {
    const data = await service.getAllObjects();
    return apiResponse(res, data);
  } catch (err) {
    next(err);
  }
});
```

ตัวอย่างท้ายหน้าแสดงการส่ง error เองเพื่อเทียบแนวคิด; template ที่แนบเป็น Express 5 จึงไม่จำเป็นต้องห่อ async route ทุกตัวแบบนี้

[R1–R4, R7] เอกสาร Express, mysql2, Node และ HTTP; ส่วนอธิบายเป็นความรู้ประกอบ

ก่อนสอบและก่อนส่ง / Checklist สุดท้าย

ลำดับซ้อม 120 นาที — เป็นแผนฝึก ไม่ใช่เกณฑ์ผู้สอน

ช่วง	ทำอะไร	สิ่งที่ต้องเห็น
0–15 นาที	Setup + DB + โครงสร้างไฟล์	เชื่อม MySQL และดู schema ได้
15–25 นาที	student-info + base path	รหัส ชื่อ key ตรงและตอบ 200
25–35 นาที	Brands สอง endpoint	รายการได้ / detail ได้ / ไม่พบเป็น 404
35–60 นาที	Products list + pagination + Summary	7 field + ตัวเลขจำนวนหน้าถูก
60–75 นาที	Product detail + JOIN	brandName และ field รายละเอียดครบ
75–100 นาที	POST / PUT / DELETE	201 / 409 / 200 / 204 / 404 ถูก
100–115 นาที	ทดสอบซ้ำ + แก้ error	success และ edge cases สำคัญผ่าน
115–120 นาที	ตรวจและบีบอัดส่ง	ชื่อไฟล์ถูก ไม่รวม node_modules

ก่อนออกจากแต่ละข้อให้ถามตัวเอง

Method และ URL ตรงหรือยัง? อ่านค่าจาก params/query/body ถูกตัวไหม? ชื่อ SQL column ตรง DDL หรือไม่? ส่ง array/object ถูกแบบไหม? DTO มี field เกินหรือขาดไหม? สถานะผิดกรณีไหม? ค่าศูนย์และข้อมูลไม่พบทำงานอย่างไร?

เช็กเนื้อหาก่อนพิมพ์ชีท

หน้า 1–2 ใช้เปิดอ้างอิงเร็ว. หน้าเฉลย A–H มี source ครบตามชื่อไฟล์. ส่วนแนวฝึกไม่ต้องนำไปปนกับโค้ดหลัก เว้นแต่โจทย์ใหม่ร้องขอ. โค้ดใน ZIP ใช้ตัวอย่างชุด 00002 เพื่อให้เห็นตำแหน่งตั้งค่าเท่านั้น

> Open book ไม่ได้แปลว่าอนุญาตทุกสื่อโดยอัตโนมัติ ให้ตรวจระเบียบของรายวิชาว่านำกระดาษ โค้ดอิเล็กทรอนิกส์ หรือเอกสารที่จัดทำด้วย AI เข้าได้หรือไม่ ในไฟล์ spec ที่ให้มาไม่ได้ระบุรายละเอียดสิทธิ์เหล่านี้

สิ่งที่ยังยืนยันไม่ได้

ยังไม่มี SQL และ official test runner จึงไม่ทราบ nullability/default/enum ทั้งหมด รูปแบบ error JSON ที่ผู้ตรวจคาด หรือคะแนนที่หายไป 15 คะแนน. ชีทแยกจุดเหล่านี้ออกจาก requirement ไว้แล้ว ไม่รับรองคะแนนเต็มจากการคัดลอกโดยไม่ตรวจไฟล์จริง

[S1–S4] เวลา 120 นาที และ §4; ลำดับซ้อม/เช็กลิสต์เป็นคำแนะนำของผู้จัดทำ

แหล่งอ้างอิงและหมายเหตุการใช้

จัดทำจากไฟล์ที่ผู้ใช้แนบและเอกสารทางการที่ตรวจเมื่อ 22 ตุลาคม 2026. เครื่องหมาย [S], [T], [R] ในแต่ละหน้าแยกแหล่งของข้อกำหนด โค้ดต้นแบบ และความรู้ประกอบ

รหัส	แหล่งข้อมูล	ตำแหน่ง / URL
S1	exam-spec-67130500001.md	ไฟล์ผู้ใช้: Kanokphon Sueby; ใช้หัวข้อ §1–4
S2	exam-spec-67130500002.md	ไฟล์ผู้ใช้: Kritsanaphong; ใช้หัวข้อ §1–4
S3	exam-spec-67130500003.md	ไฟล์ผู้ใช้: Kittikorn Phaen; ใช้หัวข้อ §1–4
S4	exam-spec-67130500004.md	ไฟล์ผู้ใช้: Kittithat Yotkh; ใช้หัวข้อ §1–4
T1	node-express-esm-template.zip	ตรวจ package.json และ src/ จริง; Express ^5.2.1, mysql2 ^3.24.3
R1	Express 5: Error Handling / API	https://expressjs.com/en/5x/guide/error-handling/
R2	mysql2: Promise Wrappers / Quickstart	https://sidorares.github.io/node-mysql2/docs/documentation/promise-wrapper
R3	mysql2: DECIMAL และ dateStrings	https://sidorares.github.io/node-mysql2/docs/documentation https://github.com/sidorares/node-mysql2/blob/master/typings/mysql/lib/Connection.d.ts
R4	Node.js: ECMAScript modules	https://nodejs.org/api/esm.html
R5	MySQL 8.4: SELECT	https://dev.mysql.com/doc/refman/8.4/en/select.html
R6	MySQL 8.4: Transaction / Locking reads	https://dev.mysql.com/doc/refman/8.4/en/commit.html https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html
R7	RFC 9110: HTTP Semantics	https://www.rfc-editor.org/rfc/rfc9110.html
R8	RFC 5789: PATCH Method for HTTP	https://www.rfc-editor.org/rfc/rfc5789.html

ความสัมพันธ์ระหว่าง source กับเฉลย

เอกสาร spec กำหนดสิ่งที่ API ต้องตอบ แต่ไม่มี source คำตอบครบชุด โค้ด A–H จึงเป็น implementation ที่เขียนใหม่ตาม requirement และโครงสร้าง template. โค้ดฝึก filter/stock/stats/transaction เป็นข้อเสนอเพิ่มเติม ไม่ใช่ข้อความที่พบใน spec

ชื่อและ Secret Key ในตารางโปรไฟล์คัดตามไฟล์ตัวอย่างเพื่อเทียบชุดเท่านั้น ไม่ควรนำข้อมูลผู้อื่นไปใช้ระบุตัวตนในข้อสอบ. ในชุดโค้ดไม่มีรหัสผ่านฐานข้อมูลจริงของผู้ใช้ และไม่รวมไฟล์ตั้งค่า IDE จาก template

> **กติกาการใช้ชีท:** Spec จริงเป็นหลัก → ตรวจ schema → เลือกเฉพาะโค้ดที่ตรงโจทย์ → ทดสอบกับฐานข้อมูลจริง → เปิดดูผลตรวจของผู้สอนก่อนส่ง

อ้างอิงไฟล์แนบและเอกสารทางการ; ชีทเพื่อศึกษา ไม่ใช่เฉลยที่รับรองโดยผู้สอน